AnimeVerse

Complete project guide: from the first React setup to the live GitHub Pages release.

This guide explains what was built, why each part exists, how the app gets anime data, how the code is organized, how it was tested, and how to maintain or redeploy it.

Repository: github.com/xhantimakinta/anime-app

Live app: xhantimakinta.github.io/anime-app/

Stack: React 19, Vite 8, JavaScript, CSS, Jikan, AniList, GitHub Pages

1. Project goals

AnimeVerse was created as a polished anime discovery experience. The goal was to make a useful app rather than a static demo: a visitor can find anime, compare titles, save personal choices, and open more information without leaving the page.

The app is intentionally frontend-only. It does not require a database, user accounts, server code, or private credentials. Public anime APIs provide the catalog and the browser stores personal library choices locally.

2. Technology choices

3. Project creation and setup

The project began as a Vite React application. Vite supplies a small HTML entry point, a React entry file, development tooling, and a production build pipeline.

- Create or open the project folder.
- Install React, Vite, ESLint, and the deployment package.
- Use src/main.jsx as the React entry point.
- Build the application in src/App.jsx.
- Keep global styles in src/index.css and app styles in src/App.css.

Install and run

```
npm install
npm run dev
```

Vite starts a local development server, normally at http://localhost:5173/. The page updates as source files change.

4. Application structure

```
anime-app/
|-- public/favicon.svg Browser tab icon
|-- src/App.jsx Main app logic and UI
|-- src/App.css Component and responsive styling
|-- src/index.css Global styles and focus states
|-- src/main.jsx React entry point
|-- index.html HTML shell and metadata
|-- vite.config.js Vite and Pages configuration
|-- package.json Scripts and dependencies
|-- README.md Repository documentation
|-- docs/project-guide.html Source for this guide
`-- docs/AnimeVerse-Project-Guide.pdf Downloadable guide
```

Important responsibilities

- App.jsx owns fetching, fallback behavior, state, derived results, and the page structure.
- App.css styles the hero panel, catalog, filters, cards, library actions, and modal.
- index.css sets the global background, typography, browser defaults, and keyboard focus outline.
- main.jsx renders <App /> into the element with the ID root.
- vite.config.js uses base: './' so assets work on a GitHub Pages project URL.

5. Data and API architecture

The catalog requests up to 22 results. Jikan is the first provider because its REST response already resembles the fields used by the interface.

If Jikan returns a non-success response, the app makes an AniList GraphQL request. The fallback asks for title, description, episodes, status, score, favorites, genres, image, ranking, and source URL.

Normalization

AniList uses a different schema and a 100-point score. normalizeAniListAnime() maps those fields into the same object shape as Jikan and divides AniList scores by 10. This lets cards and the featured panel render either provider without duplicate UI code.

6. User features

Search and loading

The search state starts with naruto. Every change starts a 250 ms timer. A later change cancels the previous timer, reducing API traffic while the user types. Loading, success, empty, and error states are represented in the interface.

Filters and sorting

Genre buttons filter normalized genre names. The sort select preserves provider relevance or orders the current list by score, provider favorites, or title.

Favorites and watchlist

Each card has a favorite action and a watchlist action. IDs are stored in browser localStorage. The My Library button filters the current result set to anime saved in either personal list.

Details modal

The Details button opens an accessible dialog with the poster, synopsis, score, episodes, status, favorite action, and original source link. Clicking the backdrop or close button dismisses it.

Statistics

The stats row derives the average score, highest-rated title, and combined provider favorites from the active results. It updates when search, filters, or library mode changes.

7. Browser storage and privacy

The app stores only two JSON arrays locally:

- animeverse-favorites
- animeverse-watchlist

There is no account system or custom backend. The saved lists are specific to the browser and device. Clearing site data, changing browsers, or private browsing can remove or hide them. Search terms are sent to a public API because that is how the catalog is retrieved.

8. Verification and quality checks

Before deployment, the project is checked with:

```
npm run lint
npm run build
```

Lint checks source code. The production build proves that Vite can transform the React code and create the deployable dist folder. The hosted response was also checked after deployment to confirm that GitHub Pages

returned HTTP 200 and served the latest bundle.

9. Git and GitHub Pages deployment

The local repository is connected to:

```
https://github.com/xhantimakinta/anime-app.git
```

The normal deployment sequence is:

```
git status
git add .
git commit -m "Describe the change"
git push origin main
npm run deploy
```

The deploy script runs npm run build first, then publishes dist to the gh-pages branch. GitHub Pages is configured to deploy from that branch at its root folder.

Because this is a project page, the Vite relative base path is important. Without base: './', generated asset paths could point to the domain root instead of /anime-app/.

10. Troubleshooting

The catalog keeps loading

Jikan and AniList are public services. They can be rate-limited or temporarily unavailable. Wait briefly and refresh. The app automatically attempts the fallback provider when Jikan fails.

The deployed page looks old

GitHub Pages and browser caches can take time to refresh. Hard-refresh the page or open it in a private window. Confirm Pages uses the gh-pages branch and root folder.

Favorites disappeared

Favorites are browser-local, not account-synced. They do not follow the visitor to another browser or device.

Posters are missing

Images come from external APIs. A provider can change or remove an image URL independently of the project.

11. Future improvements

- Add pagination or infinite scrolling for larger catalogs.
- Add dedicated detail routes and browser history support.
- Add accounts and cloud-synced libraries through a backend.
- Add automated browser tests for search, filters, favorites, and the modal.
- Add an offline cache for recently viewed results.

12. Credits and limitations

Anime metadata and images are provided by Jikan and AniList. AnimeVerse is a frontend learning project and is not affiliated with MyAnimeList, AniList, or any anime publisher. Provider result availability, rate limits, descriptions, scores, rankings, and image URLs can change independently of this repository.

Generated for the AnimeVerse project. The HTML source for this PDF is stored in docs/project-guide.html.